Лекція №9 (2 години). JavaScript ES6+: Синтаксис та структури даних.

План лекції

1. Вступ до JavaScript та стандарту ECMAScript (ES6+).
2. Оголошення змінних: `let`, `const` та проблема `var`.
3. Типи даних у JavaScript (Примітиви та Об'єкти).
4. Шаблонні рядки (Template Literals).
5. Робота з Логічними операторами та приведення типів (Type Coercion).
6. Структури даних: Масиви (Arrays) та методи перебору.
7. Структури даних: Об'єкти (Objects).
8. Деструктуризація (Destructuring) та Spread/Rest оператори.

Перелік умовних скорочень

- **JS** — JavaScript.
- **ES6** (ECMAScript 2015) — ключова версія стандарту мови JavaScript.
- **DOM** (Document Object Model) — об'єктна модель документа.
- **JSON** (JavaScript Object Notation) — текстовий формат обміну даними, заснований на синтаксисі JS.

Вступ

HTML відповідає за структуру документа (скелет), CSS — за його зовнішній вигляд (шкіра і одяг), а **JavaScript (JS)** робить сторінку живою (м'язи та нервова система). Без JS Інтернет складався б із статичних сторінок, схожих на Вікіпедію. JS додає інтерактивність: модальні вікна, слайдери, валідацію форм "на льоту", нескінченний скрол та можливість зв'язуватися з сервером без перезавантаження сторінки.

Історично JavaScript створювався у спішці (за 10 днів у 1995 році) просто як "клей" для веб-сторінок. Через це в мові було багато неочевидних, дивних архітектурних рішень. Ситуація кардинально змінилася у 2015 році з виходом стандарту **ECMAScript 6 (ES6)**. Це було найбільше оновлення мови за всю її історію.

Сьогодні "сучасний JavaScript" означає ES6 та всі наступні щорічні оновлення (ES7, ES8 тощо). Писати код старим способом (кодом зразка 2010 року) нині вважається жорстким антипатерном.

Мета цієї лекції — познайомитися з базовим синтаксисом сучасної версії JS та основними структурами зберігання даних (масивами та об'єктами).

1. Вступ до ECMAScript

ECMAScript — це офіційна стандартизована специфікація. JavaScript — це найпопулярніша *реалізація* цього стандарту.

Починаючи з 2015 року (коли вийшов стандарт ES6), нові фічі в мову додаються кожного року невеликими порціями (ES2016, ES2017 і т.д.). Тому сьогодні розробники часто використовують загальний термін **ES6+**, маючи на увазі сучасний JavaScript.

2. Оголошення змінних: `let`, `const` та `var`

Змінна — це іменоване місце в пам'яті для зберігання даних.

У старому JS змінні оголошувалися ключовим словом `var`. Це породжувало безліч помилок, оскільки `var` не має "блочної області видимості" (його видно скрізь у функції, навіть до місця оголошення — що називається Hoisting або підняття).

В ES6 з'явилися два нових, правильних способи оголошення змінних:

1. `let`: Змінна, значення якої **може** змінюватися (перепризначатися) у майбутньому. Вона існує (доступна) лише всередині тих фігурних дужок `{}`, де була створена.
2. `const`: Константа. Якщо ви присвоїли їй значення, змінити його через знак дорівнює `=` більше **неможливо**. (Однак, якщо всередині `const` лежить масив чи об'єкт, ви можете змінювати їхній *вміст*).

Золоте правило сучасного JS: Завжди використовуйте `const` за замовчуванням. Якщо ви заздалегідь знаєте, що значення змінної буде перезаписуватися (наприклад, лічильник у циклі), тоді використовуйте `let`. **Ніколи не використовуйте `var`**.

```
// Правильно:
let age = 25;
age = 26; // Працює коректно

const userName = "Ivan";
userName = "Petro"; // Помилка! TypeError: Assignment to constant variable.
```

```
// Неправильно (антипатерн):
var oldVariable = "Bad practice";
```

3. Типи даних (Примітиви та Об'єкти)

JavaScript — це мова з **динамічною типізацією**. Вам не потрібно вказувати компілятору, що в змінній лежить число або текст. Мова сама здогадається під час виконання (і цей тип може змінитися).

У JS існує 8 базових типів даних. Сім із них називаються **Примітивами** (простими значеннями), а один — структурою даних (**Об'єкт**).

Примітиви (передаються за значенням):

1. `Number` : Числа (цілі, дробові) та спеціальні числові значення (`Infinity` , `NaN` — Not a Number).
2. `BigInt` : Для роботи з надзвичайно великими числами (рідко використовується).
3. `String` : Текст (рядок). Береться у подвійні `" "` , одинарні `' '` , або зворотні `` ` ` `` лапки.
4. `Boolean` : Логічний тип. Має лише два значення: `true` (істина) та `false` (хибність).
5. `null` : Спеціальне порожнє значення. Означає "нічого", "порожньо", "значення невідоме". (Ви самі повинні присвоїти змінній `null`).
6. `undefined` : Значення не було призначено. (Якщо ви створили `let x;` але нічого туди не поклали, значення `x` буде `undefined` . Це робить сам браузер).
7. `Symbol` : Унікальні ідентифікатори об'єктів (просунута тема).

Структура (передається за посиланням): 8. `Object` : Складні структури (включають Масиви та Функції).

4. Шаблонні рядки (Template Literals)

До виходу ES6, щоб з'єднати текст (рядок) зі змінною, використовували символ плюса `+` (конкатенацію). Це було незручно. ES6 ввів **Шаблонні рядки** — рядок обгортається **зворотними (косими) лапками** (клавіша з буквою `Ё` або `~`). Змінні та вирази всередині таких лапок записуються у форматі `${змінна}` . Крім того, шаблонні рядки дозволяють легко робити багаторядковий текст без символів `\n` .

```
const name = "Марія";
const age = 22;
```

```
// По-старому (до ES6):
const oldWay = "Привіт, мене звати " + name + " і мені " + age + " роки.";

// Сучасний підхід (Template Literals):
const modernWay = `Привіт, мене звати ${name} і мені ${age} роки.`;
```

5. Логічні оператори та Приведення типів

Приведення типів (Type Coercion)

JS любить автоматично перетворювати типи даних одне в одне. Наприклад, якщо до рядка `"5"` додати число `2` (`"5" + 2`), JS не видасть помилку. Він перетворить число на рядок і "склеїть" їх. Результат: `"52"` . Але якщо відняти (`"5" - 2`), він перетворить рядок на число і видасть `3` . Ця поведінка часто стає причиною багів.

Порівняння (`==` проти `===`)

- `==` **(Несуворе порівняння):** Порівнює значення, але ігнорує типи. Якщо типи різні, він спробує їх автоматично перетворити. (`5 == "5"` дасть `true`).
- `===` **(Суворе порівняння):** Порівнює І значення, І тип даних. Без перетворень. (`5 === "5"` дасть `false`).

Правило: В сучасному JS завжди і скрізь використовуйте ТІЛЬКИ суворе порівняння `===` та несуворе нерівенство `!==` . Уникайте автоматичного приведення типів.

Логічні оператори

- `&&` **(ЛОГІЧНЕ І / AND):** Повертає `true` , тільки якщо обидві сторони (і ліва, і права) є істинними.
- `||` **(ЛОГІЧНЕ АБО / OR):** Повертає `true` , якщо хоча б одна зі сторін є істинною.
- `!` **(ЛОГІЧНЕ НІ / NOT):** Інвертує (перевертає) значення. `!true` стає `false` .

```
const isAdult = true;
const hasTicket = false;

if (isAdult && hasTicket) {
```

```
  console.log("Вхід дозволено"); // Цей код не виконається, бо hasTicket = false
}
```

6. Структури даних: Масиви (Arrays)

Масив — це впорядкована колекція даних. Уявіть собі шафу з пронумерованими шухлядами, де лежать речі. Елементи масиву розташовуються в квадратних дужках `[]` через кому. Нумерація (індексація) завжди **починається з нуля**.

Тобто `arr[0]` — це завжди перший елемент.

```
const colors = ["червоний", "зелений", "синій"];
console.log(colors[1]); // Виведе "зелений" (другий елемент)
```

Масиви в JS можуть одночасно містити будь-які типи даних (числа, рядки і навіть інші масиви або функції), але на практиці краще зберігати в одному масиві дані одного типу.

Методи масивів

Масиви мають величезну кількість вбудованих функцій (методів), які допомагають з ними працювати:

- `push()` — додає елемент в кінець.
- `pop()` — видаляє останній елемент.
- `includes()` — перевіряє, чи є елемент у масиві (повертає `true` / `false`).
- `length` (властивість, не метод) — містить кількість елементів у масиві.

ES6 додав потужні методи для перебору (циклів) по масивам, такі як `map`, `filter`, `forEach` (їх ми розглянемо детальніше на наступних лекціях).

7. Структури даних: Об'єкти (Objects)

Якщо масив — це колекція пронумерованих даних, то **Об'єкт** — це структура для зберігання даних у форматі "Ключ: Значення" (Key-Value). Вони використовуються для опису складних сутностей. Наприклад, реального користувача або товару. Створюються об'єкти за допомогою фігурних дужок `{}`.

```
const user = {
  firstName: "Олена",
  age: 28,
  isAdmin: false,
```

```
  "likes cats": true, // Якщо в ключі є пробіли, беремо в лапки
};

// Читання властивостей (через крапку):
console.log(user.firstName); // "Олена"

// Зміна значень або додавання нових:
user.age = 29;
user.city = "Львів"; // Додасться нова властивість
```

Ключем (властивістю) в об'єкті завжди є рядок (текст), а значенням — абсолютно будь-який тип даних (навіть масиви або інші вкладені об'єкти).

8. Деструктуризація та Spread оператор

ES6 ввів надзвичайно зручний синтаксис для роботи з масивами та об'єктами.

Деструктуризація (Destructuring Assignment)

Це можливість миттєво "розпакувати" (дістати) необхідні властивості з об'єкта або масиву і записати їх в окремі змінні.

```
// Без деструктуризації:
// const name = user.firstName;
// const userAge = user.age;

// З деструктуризацією (Дістань 'firstName' і 'age' з 'user'):
const { firstName, age } = user;
console.log(firstName); // "Олена"
```

Spread оператор (`...`)

Дозволяє "розсипати" (розпакувати) елементи одного масиву/об'єкта для того, щоб скопіювати їх або вставити у новий масив/об'єкт.

Копіювання та об'єднання об'єктів:

```
const defaultSettings = { theme: "dark", lang: "uk" };
const userSettings = { volume: 100 };
```

```
// Створюємо новий об'єкт, висипаючи туди все з обох
const finalSettings = { ...defaultSettings, ...userSettings };
// Результат: { theme: "dark", lang: "uk", volume: 100 }
```

Копіювання масивів:

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]
```

> ***Хінт:*** *Це сучасний і безпечний спосіб створення копії об'єкта, оскільки просте присвоєння* `const a = b` *(де* `b` *- це об'єкт) не створює копію, а лише дає нове посилання на ту ж саму комірку в пам'яті комп'ютера.*

Висновки

1. ES6 — ключове оновлення стандарту мови JavaScript, використання можливостей якого є обов'язковим у сучасній веб-розробці.
2. Змінні слід оголошувати лише через `const` (за замовчуванням) або `let`. Забудьте про `var`.
3. JS підтримує динамічну типізацію та має два види порівняння. Обов'язкова умова чистого коду — завжди використовувати суворе порівняння (`===`).
4. Шаблонні рядки (через зворотні лапки `` ` ``) — це найзручніший спосіб поєднувати текст зі змінними (`${val}`).
5. Для зберігання списків (упорядкованих наборів) використовуються масиви (`[]`), для впорядкування сутностей з властивостями — об'єкти (`{}`). Їх можна комбінувати як завгодно.
6. Деструктуризація і Spread-оператор (`...`) дозволяють писати набагато чистіший і коротший код для "витягання" даних та копіювання об'єктів.

Джерела

1. MDN Web Docs: JavaScript Guide
2. Ілля Кантор (uk.javascript.info) - Сучасний підручник з JavaScript українською мовою.
3. ES6 Features (Luke Hoban) - Коротка шпаргалка по всім фічам ES6.

Запитання для самоперевірки

1. В чому полягає критична проблема використання `var` для створення змінних, і коли замість `const` слід використовувати `let`?
2. Назвіть 5 примітивних типів даних у JS. У чому їхня глобальна відмінність від Об'єктів?
3. Що буде результатом виразу `typeof null` згідно з документацією (це відомий баг мови)? Що таке `undefined`?
4. Навіщо у HTML і JS придумали два оператори порівняння: подвійне (`==`) і потрійне (`===`) дорівнює?
5. З якого числа прийнято починати нумерацію елементів будь-якого масиву? Як отримати останній елемент масиву, якщо його довжина заздалегідь невідома?
6. Що таке об'єкт в JS? У якому форматі там зберігаються дані?
7. Створіть змінну `myString` використовуючи шаблонні рядки, яка міститиме текст "Разом: X грн", де X береться із заданої числової змінної `totalPrice`.
8. Опишіть своїми словами, що робить оператор `...` (Spread) і як його застосувати для злиття двох масивів у третій?